$f: \mathbb{N} \to \mathbb{R}$

$f \in O(g) . \exists M > 0 . \exists N \in \mathbb{N} . \forall n > N . \ f(n) \leq M g(n)$

množenje $n^3 \rightsquigarrow n \log_2 7 \rightsquigarrow n$

$$\boxed{\lim_{x \to \infty} \frac{f(x)}{g(x)} = 0 \Rightarrow f \in O(g)}$$

$\forall \varepsilon > 0 . \exists N > 0 . \ x > N . \Rightarrow \left| \frac{f(x)}{g(x)} \right| < \varepsilon$

$\Rightarrow |f(x)| < \varepsilon |g(x)|$

pokažemo je

$\exists \varepsilon > 0 . \exists N > 0 . \forall n > N . \ f(n) \leq \varepsilon g(n) \Rightarrow f \in O(g)$

$\varepsilon = 1$

$o(1) \subset O(\ln(\ln(n)))$

$O(\log_2(n)) = O(\log_{10}(n))$

$\log_2(n) = \frac{\log_{10} n}{\log_{10} 2} \in O(\log_{10}(n))$

$\log_{10}(n) = \frac{\log_2 n}{\log_2 10} \in O(\log_2(n))$

kolena

- że cało przejdzie (przebiegam po kolorze)
- zdaj że po kolorze jeszcze na fakty
- sortowanie ma dwa długie krawędzie

$$O(1) \subset O(\log(\log n)) \subset O(\log_2(n)) = O(\log_{10}(n))$$

$$\subset O(\sqrt{n}) \subseteq O(\log(n!)) \subseteq O(n \log n) \subseteq O(3^{\ln n})$$

$$\boxed{O(\log_a(n)) \subset O(n^{\varepsilon})} \qquad \subseteq O(n^2) \subseteq O(2^n)$$

$$\subseteq O(2^{2\log n})$$

$$\log(n!) \leq \log(n^n) = n \log n$$

$$O(\log(\log(n))) \subseteq O(\log n)$$

$\log(n) < n$ $\qquad$ $\ln \log$ inj nerównościowa

$$\log(\log(n)) \leq \log(n)$$

$$\log(n) \in O(\sqrt{n})$$

$$\lim_{n \to \infty} \frac{\log n}{\sqrt{n}} \overset{L'H}{=} \frac{1}{n} \frac{\sqrt{n}}{\frac{1}{2}} = \lim 2 \frac{1}{\sqrt{n^3}} = 0 \Rightarrow \log(n) \in O(\sqrt{n})$$

$$3^{\ln(n)} = 3^{\frac{\log_3(n)}{\log_3 e}} = = n^{\frac{1}{\log_3 e}} = n^{\varepsilon} > 1$$

$$\sqrt{n} < n < n \log n$$

$$O(n \log n) = O(\log n!)$$

$$\log n! = \sum_{i=1}^{n} \log i$$

$$\log n! > \log\left(\frac{n}{2}\right)^{\frac{n}{2}} = \frac{n}{2}(\log n - \log 2) \cong n($$

$$n \log n < n^2$$

$$n \log n \leq n \cdot n \leq n^2$$

$$O(n^2) \subseteq O(2^n) \subseteq O(2^{n \log n} \subseteq O(n!)$$

$$2^{\log n^n} = 2^{\frac{\log_2 n^n}{\log_2 10}} = n^{\log_2 10} n$$

seznam: n

append: $O(1)^*$

delete(i) $O(n)$ $(n-i)$

copy: $O(n)$

update(i): $O(1)$

get(i): $O(1)$

find(a): $O(n)$

add: $O(n)$

to nes tipična zanim	
add(i)	$O(n)$
add(0,n)	$O(n)O(1)$
delete(i)	$O(n)$
delet(0,n)	$O(n)O(1)$
get(i)	$O(1)$
search(x)	$O(n)$

to nes tipična zanim	čas	pro star
add(i)	$O(n)$	$O(1)$
add(0,n)	$O(n)O(1)$	$O(1)$
delete(i)	$O(n)$	$O(1)$
delet(0,n)	$O(n)O(1)$	$O(1)$
get(i)	$O(1)$	$O(1)$
search(x)	$O(n)$	$O(1)$

Slovar

$O(1)$ vse

algoritem za iskanje maximuma

```
m = seznam[0]
for c in seznam[1:]
    if c > m => m = c
return m
```

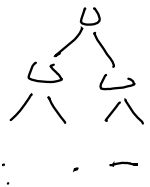
$O(n), O(1)$

```
seznam n
seznam.sort
seznam[-1]
```

$O(n \log n)$ $O(1)$

$$\sum_{i \in N}^{n} i = O(n^2)$$

Rekurzivno: seznam na dva dela



$\Big\}$ $\log n$ globina

čas je lahko $O(\log n)$ z prostorom n procesorjev

lahko se je tudi $O(\frac{n}{\log n})$ prostora inje

še vedno hitreje

Bitne operacije

- bitand &
- bitor |
- bitxor ^
- bitnot !, ~
- bitshiftleft <<
- bitshift right >>

v jezicih
&&
||
!=
not

z: 1 1 1
g: 1 0 0 1

$$\begin{array}{r} 111 \\ 1001 \\ \hline 1 \end{array} \&$$

$$\begin{array}{r} 0111 \\ 1001 \\ \hline 1111 \end{array} |$$

$$\begin{array}{r} 111 \\ 1001 \\ \hline 1110 \end{array} ^$$

$$! \begin{array}{r} 111 \\ \hline 0 \end{array} \qquad ! \begin{array}{r} 1001 \\ \hline 110 \end{array}$$

$$<< \begin{array}{r} 111 \\ \hline 1110 \end{array} \qquad << \begin{array}{r} 1001 \\ \hline 10010 \end{array} \qquad g >> 2 \begin{array}{r} 1001 \\ \hline 10 \end{array}$$

$$a \oplus b \oplus b = a$$

a \ b	0	1
0	$0 \oplus 0 = 0$	$1 \oplus 1 = 0$
1	$1 \oplus 0 = 1$	$0 \oplus 1 = 1$

a...bit

$$
\begin{array}{l}
\quad a_n \ldots\ldots a_2 a_1 a_0 \\
\& \; [0 \ldots 01\ 0 \ldots 0 \ldots 0] \leftarrow \text{maske} \\
\hline
\quad 0 \ldots\ldots a_i \ldots\ldots 0
\end{array}
$$

if $\underbrace{(a \,\&\, \text{mask})}_{>0 \text{ zato True}}$

maske $= 1 \ll i$

$$
\begin{array}{l}
1.)\ [0 \ldots\ldots 0] \\
\hline
\sim [1 \ldots\ldots\ \ 1] \\
\hline
\gg n-i\,[0 \ldots\ldots 1 \ldots 1]
\end{array}
$$

$\sim((\sim 0) \ll i)$

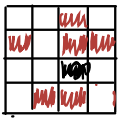
$(1 \sim i) - 1$

funkcja spełnia a, i

$a_n \ldots a_1 0 \rightarrow a_n \ldots 1 \ldots a_1 a_0$

```
def f(a,i):
    mask := 1<<i
    return a | mask

def f'(a,i)
    mask := ~(1<<i)
    return a & mask
```



10 miejsc

n = 0 0 1 0 1 0 1 1 0 0 0 0 0 1 1 0 1 0 1 0

lewo: n-1
dsno: n+1
gor: n-u
dol: n+u

```
def lewopojj(a)
    maskp = (1<<u)-1
    p = a & maskp
    mask = ~(1<<20-p_10+1)
    return (a & mask) - 1
```

a = a & mask
a = (a>>u)<<u
return a | p

```
def fib(n):
    prv = 1
    drv = 1
    for i in range(n)
        drv = drv'
        drv += prv
        prv = drv'
    return drv
```

$O(drv) = 1{,}6^i = 1{,}6^n$

↖ velikost

st bitov za drv $= \log_2 1{,}6^i$

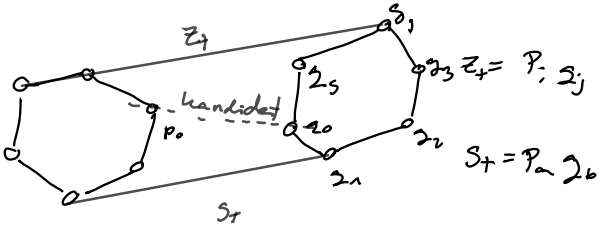
$= O(i)$

assign $= O(n)$

$$\sum v_1 + v_2 + v_3 = \sum_{i=1}^{n} 3O(i) = O(n^2)$$

TRIE – prefix tree

España

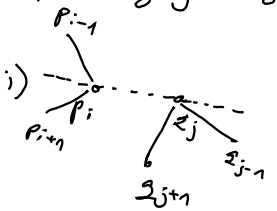


Združena avojnica ba nato ttoto

Kako do z_+

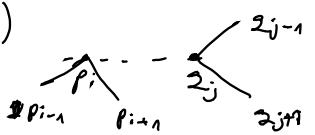
$z_+ = p_0 q_0$

while z_+ ni zagrnja tangenta

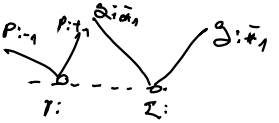


V tem primeru nadaljujemo

$p_i = p_{i-1}$ $(i = i-1)$



$j = j+1$



Ko je zagrnja tangenta, dvignemo oba

$i += 1$ $j += 1$

Bla bla bla

Če pozicija ni naredi $+=1$

Predstave grafa

- Matrika sosednjosti $A=[a_{ij}]$; $a_{ij} = \begin{cases} 1; & i \text{ in } j \text{ povezana} \\ 0; & \text{sicer} \end{cases}$
- slovar { vozlišče : [povezave do vozlišča] }

```
{ v: [u for u in next(v)] for v in G}
```

opomba: če vozlišča označimo od 0 do n-1, namesto slovarja uporabimo seznam

ideja DFS:

1. izberemo začetno vozlišče in ga shranimo v obiskane
2. izberemo sosede, ki še ni bil obiskan, označimo obiskan
3. pomaknemo se v sosedna

ideja BFS:

1. izberemo začetno vozlišče in ga damo v obiskane
2. obiščemo vsakega soseda in jih damo v obiskane, ter naredimo seznam vseh sosedov sosedov
3. obiščemo vse sosede sosedov ..... ponovi 2.

```
from collections import deque
```

double ended que

O(1) add(x) (da na desno)

O(1) pop(x) (vzam: desno)

O(1) popleft(x) (vzam: leo)

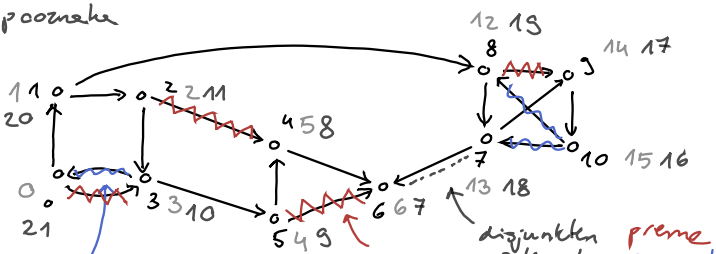
```
def DFS(G,s):
    n = len(G)
    obiskani = [False]*n
    sklad = deque([S])
    while sklad:
        v = sklad.pop()
        if not obiskani[V]
            for u in G[V]:
                if obiskani[u] == False:
                    sklad.add(u)
            obiskani[V] = True
```

standardna struktura

```
def BFS(G,s):
    n = len(G)
    obiskani = [False]*n
    vrsta = deque([S])
    while vrsta:
        v = vrsta.popleft()
        if not obiskani[V]
            for u in G[V]:
                if obiskani[u] == False:
                    vrsta.add(u)
            obiskani[V] = True
```

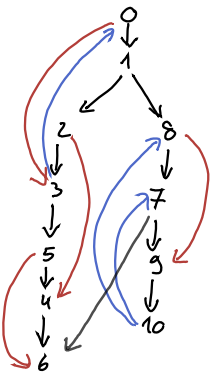
- Predoznake
- pooznake

DFS

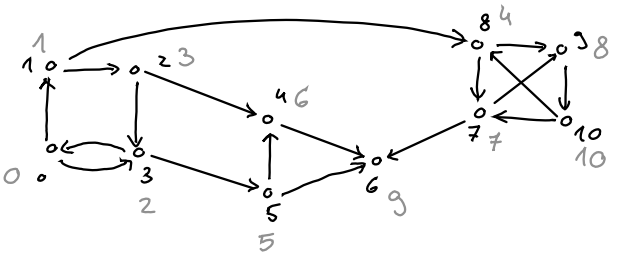


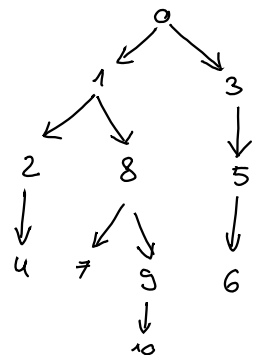
preme
poravne m
preone

~~BFS~~

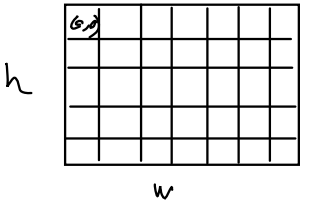


BFS





Razvij pristop za delo z grafi, ki so mreže



vozlišč = h·w (celice)

4-sosednost

$(i,j) \sim (i \pm 1, j)$
$(i,j) \sim (i, \pm j)$

```
smeri := [(1,0), (-1,0), (0,1), (0,-1)]
def get-sosedi(i,j):
    return [(i+a, j+b)  for (a,b) in smeri]
```

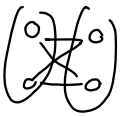
G neusmerjen graf

Algoritem Da če graf dvodelen, ne sicer

$V(G) = A \sqcup B$ -bipartacija na dva dela, tako da med

$\Leftrightarrow$

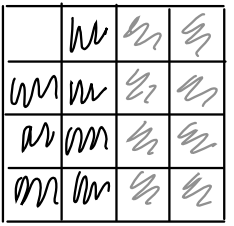
kako lahko pobarvamo $V(G)$ tako, da vozlišča vedno dobijo drugo barvo.

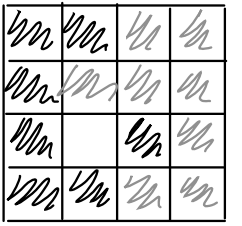


```
from collections import deque
def je_dvodelen(G):
    n = len(G)
    barve = [None]*2        = obiskani
    vrsta = deque([s])
    barva[0] = 0
    while not vrsta.empty:
        u = vrsta.popleft()
        trb = barva[u]
        for v in G[u]
            if barva[v] == None
                vrsta.push(v)
                barva[v] = trb   % 2
            elif barva[v] == trb
                return False
    return True
```

```
def pot_do_najk (G)
    n=len(G)
    razdalje=[None]*n
    pred = [None]*n
    razdalja[0]=0
    vrsta.add(0)
    while not vrsta.empty
```

3 Mreža 4x4 7 rdečimi 8 modrimi polji 1 prazno





Zanima nas najmanjše število potez iz A v B

poljubni pozicija

Vsaka konfiguracija je svoje vozlišče

pozicija bele — 4 biti

rdeče in modre — 16 bitov

Sosedi:

razen na robu

pozicija bele $(\pm 1 \vee \pm 4) \wedge$ pa vsak isto kam

razen v pozicijah bele, tam pa mora biti ista barva

28.11

zwei: pomembno

5. 12

Dijkstrov algoritem

vhod: G usmerjen/neusmerjen graf,
povezano utežen, uteži so nenegativne,
$w: E(G) \longrightarrow \mathbb{R}_+$

za začetno vozlišče s (opcija še končno vozlišče)

izhod ⇒ razdalje najkrajše poti od s od vseh ostalih vozlišč

- zapomnemo v s $d(s)=0$
- pogledamo sosede od s in izberemo tistega, ki je „najbližji" $d(u)=w(u)$
- ostale sosede damo na prioritetno vrsto v obliki $(d(s)+w(v), v)$
- nato iz vrste

za dvojiško kopico

$$O(|V|+|E|(\text{zahtevnost dodajanja} + \text{zahtevnost brisanja}))$$

$|V|$: kolikokrat je na prioritetni vrsti

zahtevnost dodajanja: $\log(E)$, zahtevnost brisanja: $\log(E)$

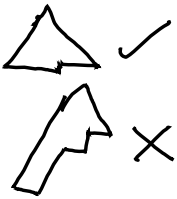
PQ seznam: dodajanje $O(1)$, brisanje $O(E)$

$$= O(|V|+|E|+E(\log E)) = O(E\log E + V)$$

Lahko tudi $O(V^2)$

Dvojiška kopica

- levo poravnano dvojiško drevo
- lastnost kopice: vrednost vsakega starša ≤ min vrednosti obeh otrok



V korenu je najmanjši element, ampak to ni iskalno drevo

Osnovne operacije

dodaj (x)

briši (izbriše minimum)

dodaj:

dodamo ga na zadnje mesto in dvigujemo gor, dokler ne zadošča lastnosti kopice

briši:

vzamemo koren in ga izbrišemo.

Najbol spodnji desni element prestavimo v koren in potem ga boublamo dol

12.12

Bellman-ford

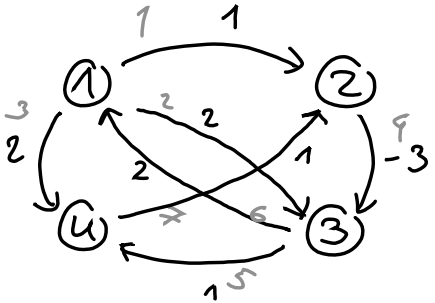
vhod: graf G (usmerjen, utežen), zač. vozlišče
↳ dovolimo negativne povezave
↳ ne dovolimo negativnih ciklov

izhod: seznam d_z dolžin najkrajših poti od s do ostalih vozlišč.

Ideja: (n-1)-krat relaksiramo vsako povezavo

```
def f(G,s):
    d = [∞]*n        n... št vozlišč
    d[s] = 0
    for _ in range(n-1):
        for (u,v,w) in E(G):
            if d[v] > d[u]+w:
                d[v] = d[u] +w
    return d
```

} relaksacija povezave



	0	1	2	3	4
1	0	0	0	0	0
2	∞	1	1	1	0
3	∞	2	-2	-2	-2
4	∞	2	2	-1	-1

po tem vemo, da je nekje negativni cikel

Tu bi se moralo končati

2) Floyd Warshallov algoritem

a) $D_k(i,j)$ ..... dolžina najkrajše poti od i do j, k: vsebuje le vozlišča $1, \dots, k$
notranja

$$D_0(i,j) = \begin{cases} w(i,j) & ; \exists (i,j,w) \\ \infty & ; \text{sicer} \end{cases}$$

$$D_k(i,j) = \min\{D_{k-1}(i,j), D_{k-1}(i,k) + D_{k-1}(k,j)\}$$

$D_n(i,j)$ je najkrajša pot

če je $D_n(i,j) < 0$ potem Graf vsebuje negativen cikel

čas je $O(n^3)$ n... št vozlišč

Velika igra z odprtim svetom ima na različnih lokacijah trgovice, kjer lahko kupujemo in prodajamo surovine

npr.: 10 železa za 1000 lesa

Ker se trgovice nahajajo na različnih delih sveta, kjer so ptičkega mleka ali kakšne rastline so lahko manjkali tudi razlogi

Surovine X bi rad zamenjal za čim več surovine Y. Kako bi to naredil?

Vozlišča so vse možne surovine

$x_1 \to x_2 \Leftrightarrow \exists T.\ T(x_1, x_2)$

T... trgovina

$\mathcal{T}$... množica trgovin

$$w(x_1, x_2) = \max_{T \in \mathcal{T}} \left\{ \begin{array}{l} r(x_1, x_2) \text{ manjkajoči razlog} \\ r(x_1, x_2) \end{array} \right.$$

iščemo pot od X v Y kjer je produkt uteži na povezavah čim večji

$w(x_1, x_2) \Leftrightarrow -\log(w(x_1, x_2))$

Problem minimalnega vpetega drevesa

$G(V,E)$ utežen graf (neusmerjen)

w_e utež

Iščemo vpeto drevo T, da je vsota $\sum_{e \in T} w_e$ najmanjša možna

Kruskalov algoritem

$[w_{e_1}, \dots, w_{e_m}]$ ... sortirane povezave

začnemo z gozdom kjer je vsako krajišče svoje drevo

iščemo krajišči povezave e_i iz različnih dreves ta povezavo dodamo v gozd in s tem združimo te dve drevesi; ice ste iz istega drevesa preskočimo povezavo

Unija z rangom

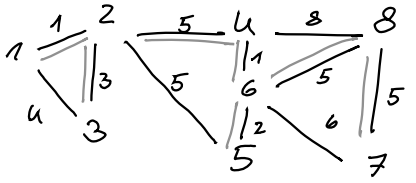
... podatkovna struktura, ki učinkovito opravlja te operacije

naredi(x) ... naredi novo množico z elementom x

najdi(x) ... vrne množico v kateri se nahaja x

Vsako množico predstavi z enim predstav.

združi(x,y) ... naredi disjunktno unijo najdi(x) in najdi(y)



(1, (1,2)), (1,(4,6)), (2,(5,6)), (3,(2,3)), (4,(1,3)),
(5, (2,4)), (5,(6,8)), (5,(7,8)) (5,(2,5)), (6,(6,7))

Čez Kruskal: $O(m \log m) + m\,O(\log n) + n\,O(\log n)$
ma najdi

m ... povezave $|E|$
n ... vozlišča $|V|$

Kompresija poti

Ko na koncu pokličemo najdi 5 ugotovimo da najdi(5) ≠ starš(5)
Potem se splača za starša(5) nastavit najdi(5)

1. izp-t 23 24

Razdelimo seznam na 3 dele.

Naj pogostejši element mora biti najpogostejši v vsaj dveh od treh delov. Lahko je tudi tretja polovica v enem, vedno v enem bo gotovo najpogostejši

```
def seznamanje(A):
    A1 = A[0 : n//3]
    A2 = A[n//3 : 2n//3]
    A3 = A[2n//3 :]
```

$\Big\}\ O(1)$

$3 \cdot T(\frac{n}{3})$ `mbi = [seznamanje(Ai) for i in {1,2,3}: if not False]`

```
    if mbi = []: return False
    a = mbi[0]
    if a ∈ mbi[1:2]: return a
```

$O(\frac{2n}{3})$ `if (A2 ∪ A3).count(a) ≥ n/6 : return True`

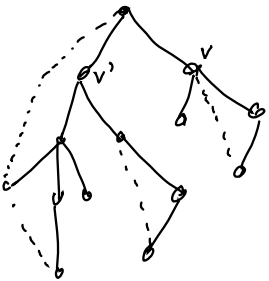
$= O(n)$

$a = 3$ $\quad b^d = 3 \Rightarrow O(n^1 \log(n))$

$b = 3$

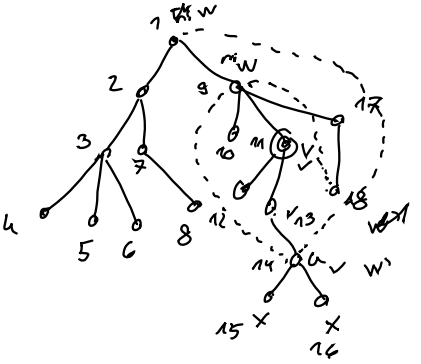
$d = 1$ $\quad$ po krovnem izreku

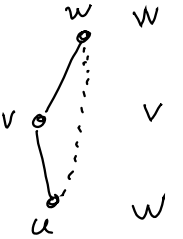
iz potence 3 $\quad d=1 \quad n^{\log_3(3)} = O(n)$



$\Rightarrow$ Recimo da je v prerezno.
Recimo da za vsakega otroka obstaja potomec, ki je povezan na predhnika od v.
Potem $\exists$ pot iz korena do vsakega potomca v L ne gre čez v, kar lahko prideno do vsakega otroka od v torej v ni prerezno.

$\Leftarrow$ Recimo da $\exists$ otrok katerega potomci nimajo povezave nazaj pokaže je dokaz. Na ta ne pridemo da s tem da prenočimo v torej je prerezno.



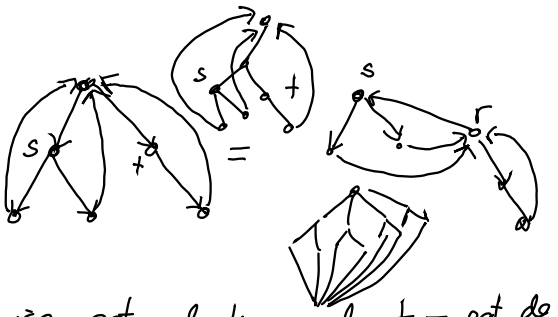


Vozlišča so urejena po pred od največ. do najmanjšega

Gremo po puščicah največjega vozlišča

low(n) = min(prev (povratne povezave))

for



Najkrajša pot od korena do t = pot do t

t potomec s ⇒ pot do t - pot do s

t na drugem drevesu ⇒ pot s do r + pot do t

za vsako vozlišče izračunamo najkrajšo razdaljo od korena. Ker je to drevo je to lahko narediti v $O(V)$

Zraven se gledamo ali je t potomec s.
če je, lahko enostavno to ugotovimo in vrnemo $w(t)-w(s)$

Drugače bomo morali poti do korena sestavit tako, da bo vse od s do r

to lahko izvedemo z dfs ali bfs v $O(V)$

potem moramo še najti kje je pot najkrajša

spet lahko uporabimo povezave, ki povezujejo največ

$2\cdot(n-2)+(n-1) \in O(n)$

potem preiščemo pot od r do t

je $O(n)$ torej lahko olajšamo

Mogoče uteži?

da je smiselno pot poiskati

mogoče

npr. že

dijkstra bela verjetno v

$O((V+E)\log(V))$ z uporabo vrste

ker je $O(n \log n)$

ali n: inteligentno celo morda da

bomo vedno morali dodati

če t potomec c je

če pa ni, pa moramo iti pot čez korenu

torej r.

R \ C	1	3	0	2	1
1	1	0	0	0	0
2	0	1	0	1	0
3	0	1	0	1	1
1	0	1	0	0	0
0	0	0	0	0	0

	0	0	0	1	0
0	1	1	1	2	1
1	.	.	.	.	1
0					
0					
0					
5	1	1	1	1	1

for r in R
    for c

	3	1	1	1	0
3	1	1	1		
2	1			1	
1	1				
0					

mora R in C (zgornjima si permutacija) to je lahko v $O(n \log n)$
Če dobim permutacijo matrike dobim
kdi drugo (vemo iz algebre 1)
potem to pomeni prvice pot kam
uniči nekaj je stopnja stolpca
in prvice pri stopnje na 0, gledamo naprej
samo ta matrika $\begin{bmatrix} \vdots \\ C \\ \vdots \end{bmatrix}$ C ne zviša se
Ta pride na $k-1$

$\text{čas} = (2 \cdot n(\log n) + 0n \cdot n = n^2 + \text{ogsta}$
$\uparrow$ sort

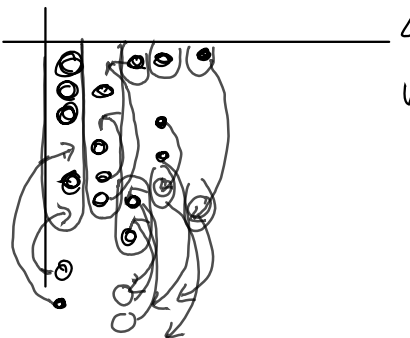
~~prav.nost~~

pravilnost? ngor pri $[i,j]$ je to $\pi(\pi(i)\pi(j))$
Ta ne bistveno drugače

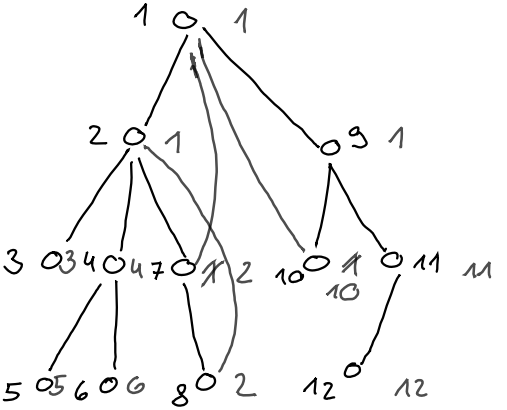
Če nogdemo prem drugo
Ali bolo vrne false tudi

Torej mora ze na neki vrstici prende
skozi bare zel .bi .ddel vse svoje

Pravimo da obstaja kdo prestrizgo
Ali je ten možje
prem je bolje kreative
p. ct. bolje ga ima



bil manjkaj
vmesne
vrstici.


1 1
2 1
9 1
3 3
4 4
7 7
1 2
10 1
10
11 11
5 5
6 6
8 2
12 12